ROS2

DAY 6 – Manual 3

Parameters – Implementation (with Dynamic Callback)

Goal

Create a ROS 2 Python node that:

- Has one parameter → number_param
- Publishes that number to a topic /number
- Reacts immediately when parameter is updated (dynamic callback)
- Allows changing the parameter at runtime using the terminal

---

1. Start Fresh – Create a New Workspace

Step 1 — Open a new terminal

```
mkdir -p DAY-06/ros2_ws/src
cd DAY-06/ros2_ws
colcon build
source install/setup.bash
```

---

2. Create a ROS 2 Python Package

Step 2 — In the same terminal

```
cd src
ros2 pkg create --build-type ament_python param_pkg
```

This creates:

- package.xml
- setup.py
- param_pkg/

3. Create the Parameter + Publisher Node (WITH CALLBACK)

Step 3 — Open the Python file

```
cd param_pkg/param_pkg
nano number_node.py
```

Paste the updated code below

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Int32
from rcl_interfaces.msg import SetParametersResult

class NumberParamNode(Node):
    def __init__(self):
        super().__init__('number_param_node')

        # Declare parameter
        self.declare_parameter('number_param', 10)

        # Publisher
        self.publisher = self.create_publisher(Int32, 'number', 10)

        # Add callback for dynamic parameter update
        self.add_on_set_parameters_callback(self.param_update_callback)

        # Timer
        self.timer = self.create_timer(1.0, self.timer_callback)

        self.get_logger().info("Node started. Publishing number_param every second.")

    # ===== PARAMETER CALLBACK (NEW PART) =====
    def param_update_callback(self, params):
        for param in params:
            if param.name == "number_param":
                self.get_logger().info(
                    f"Parameter updated! New number_param = {param.value}"
                )

        # Must return success
        return SetParametersResult(successful=True)
```

```
    # Timer publishes the parameter value
    def timer_callback(self):
        value = self.get_parameter('number_param').value
        msg = Int32()
        msg.data = value
        self.publisher.publish(msg)
        self.get_logger().info(f"Published: {value}")


def main(args=None):
    rclpy.init(args=args)
    node = NumberParamNode()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Save → Ctrl + O, Enter
Exit → Ctrl + X

---

4. Make Executable Entry Point

Step 4 — Edit setup.py

```
cd ..
nano setup.py
```

Find:

```
entry_points={
    'console_scripts': [
    ],
},
```

Replace with:

```
entry_points={
    'console_scripts': [
        'param_node = param_pkg.number_node:main'
    ],
},
```

Save and exit.

---

5. Build the Workspace

Step 5 — Open a new terminal

```
cd DAY-06/ros2_ws
colcon build
source install/setup.bash
```

---

6. Run the Node

Step 6 — Open a new terminal (Terminal 1)

```
ros2 run param_pkg param_node
```

Expected output:

```
Node started. Publishing number_param every second.
Published: 10
Published: 10
...
```

---

7. View the Topic Data

Step 7 — Open Terminal 2

```
ros2 topic echo /number
```

Output:

```
data: 10
data: 10
...
```

---

8. Change Parameter in Real-Time (Dynamic Callback Works Here)

Step 8 — Open Terminal 3

```
ros2 param set /number_param_node number_param 99
```

Terminal 1 now shows:

```
Parameter updated! New number_param = 99
Published: 99
Published: 99
...
```

Terminal 2 shows:

```
data: 99
data: 99
...
```

---

9. Override Parameter at Startup

Step 9 — In Terminal 1

```
ros2 run param_pkg param_node --ros-args -p number_param:=42
```

Output:

```
Parameter updated! New number_param = 42
Published: 42
Published: 42
...
```

---

10. SUMMARY

- Creating workspace
- Creating ROS 2 Python package
- Declaring parameters
- Publishing parameter values
- Dynamic parameter update callback

- Viewing topic data
- Overriding parameters at runtime
- Overriding parameters at startup
